A CASE-STUDY BASED REPORT

ON

A Case Study on Multi-Class Disease Diagnosis Using Multinomial Logistic Regression

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in
COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

A.Y.: 2026–27

BY

M. Shiva | Roll No.: 2520090135

P. Bhavan | Roll No.: 2520090146

Batch : 8

Under the Esteemed Guidance of

Ms. K. Chandusha
Assistant Professor, CSIT

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

1. Introduction

  1.1 Background

  1.2 Motivation for the Project

  1.3 Overview of the Work Carried Out

  1.4 Scope and Responsible-Use Statement

2. Problem Statement

  2.1 Case-Study Scenario

  2.2 Central Case-Study Question

  2.3 Formal Problem Definition

  2.4 Objectives that Guide the Study

  2.5 Assumptions and Limitations

3. Dataset and Exploratory Data Analysis

  3.1 About the Dataset

  3.2 Purpose of EDA

  3.3 Preparation Steps Actually Performed

  3.4 EDA Exploration – Implementation

  3.5 Summary of EDA Findings

4. Algorithm Selection and Implementation

  4.1 Why Multinomial Logistic Regression

  4.2 Mathematical Formulation

  4.3 Model Training and Regularization

  4.4 Evaluation and Benchmarking

  4.5 Explainability and Odds Ratios

5. System Design and Web Application

  5.1 Architecture and Workflow

  5.2 Web Application Features

  5.3 REST API and Batch Prediction

  5.4 Technical Stack

6. Case-Study Analysis and Discussion

  6.1 Example Diagnostic Scenario

  6.2 Interpretation of Model Output

6.3 Benefits

6.4 Limitations and Responsible Use

7. Conclusion

References / Source Materials

Signature of the Team Members

Abstract

Healthcare systems generate large volumes of patient information, including demographic details, vital signs, clinical symptoms, and laboratory-related measurements. Traditional diagnostic workflows may depend heavily on manual interpretation, which can be time-consuming and susceptible to human error. This case study presents an academic Clinical Decision Support System (CDSS) that applies Multinomial Logistic Regression (MLR), also known as Softmax Regression, to multi-class disease diagnosis.

The project models the probability distribution over 18 disease categories using 47 clinical features. The target pathologies cover cardiovascular, respiratory, infectious and tropical, gastrointestinal and hepatic, endocrine and metabolic, nephrology and urology, and neurology/autoimmune conditions. The implementation combines preprocessing, feature selection, model training, evaluation, explainability through model weights and odds ratios, and a Flask-based web interface for single-patient and batch prediction.

The project documentation reports an independent stratified test set of 1,080 patient cases. The tuned Multinomial Logistic Regression model achieved 98.98% accuracy, 0.9899 macro precision, 0.9898 macro recall, 0.9898 macro F1-score, and 0.9999 macro ROC-AUC, outperforming the benchmark K-Nearest Neighbors, Random Forest, and Gaussian Naive Bayes models reported in the project README.

This report follows the supplied case-study template and uses the project presentation, abstract, README, application code, and the provided demonstration cohort as source material. It is an academic decision-support prototype, not a replacement for qualified medical diagnosis or clinical judgment.

1. Introduction

1.1 Background

Machine Learning provides statistical methods for discovering patterns in data and using those patterns for prediction. In healthcare, patient records can contain many interacting measurements and symptoms. When several diseases are possible at the same time, a binary classifier is insufficient because the task is naturally multi-class.

The supplied project addresses this classification problem with Multinomial Logistic Regression. Unlike standard binary logistic regression, the multinomial formulation produces a probability for every target class. This makes the model suitable for differential-diagnosis-style ranking in which a top prediction can be accompanied by alternative disease probabilities.

1.2 Motivation for the Project

- Healthcare systems generate patient data at a scale that can outpace manual review capacity.
- Traditional interpretation can be slow and may introduce avoidable human error.
- A unified multi-class model can compare multiple disease categories from the same patient feature vector.
- Probabilistic outputs can provide a ranked set of possible diagnoses instead of a single unexplained label.
- An interpretable statistical model is useful for an academic CDSS prototype because feature influence can be examined through model coefficients and odds ratios.

1.3 Overview of the Work Carried Out

The work follows a structured machine-learning pipeline: patient data are prepared, relevant features are selected, the Multinomial Logistic Regression model is trained and tuned, and the resulting model is evaluated against benchmark algorithms. The trained model is then integrated into a Flask web application. The

application provides interactive patient assessment, batch CSV processing, analytics, a medical knowledge base, and REST APIs for programmatic prediction.

The project README describes serialized model parameters, preprocessing state, label-encoder state, model metadata, a Jupyter notebook, a vectorized machine-learning core, a training orchestrator, and inference/explainability utilities.

1.4 Scope and Responsible-Use Statement

The scope of this case study is academic exploration of multi-class disease classification and the engineering of a clinical decision-support prototype. It demonstrates how a machine-learning model can organize patient information and return probabilistic diagnostic suggestions. It is not presented as an autonomous medical diagnosis system. Real clinical use would require appropriate clinical validation, representative external datasets, privacy and security controls, regulatory review, monitoring, and qualified healthcare professional oversight.

2. Problem Statement

2.1 Case-Study Scenario

Consider a healthcare setting in which a patient record contains demographic information, vital measurements, laboratory-related values, and symptoms from several body systems. The clinician may need to distinguish among cardiovascular, respiratory, infectious, endocrine, gastrointestinal, renal, neurological, and autoimmune possibilities. Reviewing all combinations manually can be difficult when many candidate conditions share overlapping symptoms.

The proposed case-study solution receives the available patient features, applies the stored preprocessing and trained model parameters, and produces a ranked multi-class prediction. The web application can perform the same task for individual patient data or for a CSV cohort, supporting both demonstration and research-oriented analysis.

2.2 Central Case-Study Question

Can an interpretable Multinomial Logistic Regression model use a structured set of patient demographics, vital signs, biomarkers, and symptoms to classify records across 18 disease categories with high predictive performance while providing probabilistic and feature-level explanations suitable for an academic clinical decision-support prototype?

2.3 Formal Problem Definition

Let x represent a patient feature vector containing 47 clinical features and let Y be the disease class. The task is to learn a function that maps x to the probability of each of K = 18 disease classes. The predicted class is the class with the highest posterior probability.

$$z_k = w_k^{\top} x + b_k$$
$$P(Y = k \mid x) = \exp(z_k) / \Sigma_j \exp(z_j), \quad k = 1, \ldots, 18$$

The project uses an L2-regularized categorical cross-entropy objective. Regularization penalizes excessively large model weights and is intended to improve generalization while retaining a linear, interpretable model structure.

2.4 Objectives that Guide the Study

1. Automate classification of patient records into multiple disease categories.
2. Improve diagnostic classification accuracy through statistical learning and optimized model configuration.
3. Provide probability scores that allow alternative diagnoses to be considered.
4. Provide an interpretable mechanism for understanding feature influence.
5. Build a scalable prototype that can support single-patient assessment and batch cohort analysis.
6. Expose the model through a practical web interface and REST APIs.

2.5 Assumptions and Limitations

The case study is based on the data and project documentation supplied with the assignment. The README describes a 5,400-record dataset across 18 classes and a stratified test set of 1,080 cases. The attached demonstration CSV contains 20 records and is used here only for supplementary exploratory inspection.

- The project documentation does not provide a full clinical validation protocol or prospective hospital evaluation.
- The reported metrics are those documented by the project; they should not be interpreted as proof of real-world clinical performance.
- The attached demonstration cohort is small and contains no explicit disease-label column, so it is not used to recompute model accuracy.
- Specific EDA statistics for the full 5,400-record training dataset are not included in the supplied report materials; therefore this case study does not invent such values.
- Clinical decisions should remain under qualified professional supervision.

3. Dataset and Exploratory Data Analysis

3.1 About the Dataset

The project README identifies the main dataset as disease_diagnosis_dataset.csv with 5,400 patient records across 18 classes. The model is described as using 47 clinical features. These include demographic variables, continuous measurements such as age, BMI, blood pressure, heart rate, body temperature, oxygen saturation and fasting blood sugar, together with binary symptom indicators and categorical variables such as gender and smoking history.

Disease Group	Target Pathologies
Cardiovascular	Essential Hypertension; Coronary Artery Disease (CAD / Angina)
Respiratory	Bronchial Asthma; COPD; Bacterial Pneumonia
Infectious & Tropical	COVID-19 Viral Infection; Dengue Fever; Malaria
Gastrointestinal & Hepatic	GERD; Acute Gastroenteritis; Fatty Liver Disease
Endocrine & Metabolic	Type 2 Diabetes Mellitus; Hypothyroidism; Hyperthyroidism
Nephrology & Urology	Chronic Kidney Disease (CKD); Acute Urinary Tract Infection (UTI)
Neurology & Autoimmune	Migraine with Aura; Rheumatoid Arthritis

3.2 Purpose of EDA

Exploratory Data Analysis is used to understand the structure and quality of the patient data before model training. For this project, EDA supports inspection of variable types, ranges, categorical values, symptom indicators, and the relationship between patient attributes and disease classification. It also provides a basis for identifying preprocessing requirements.

3.3 Preparation Steps Actually Performed

- Numerical clinical features are handled as quantitative model inputs.
- Categorical variables are represented through the preprocessing state documented by the project; the stored state includes a Standard Scaler and OneHot Encoders.
- Disease labels are represented using a stored label-encoder state.
- The model is trained for a multi-class target with 18 classes.
- The workflow proceeds from preprocessing to feature selection, model training, and evaluation.
- Model parameters, preprocessing state, label encoding, and metadata are stored for inference.

3.4 EDA Exploration – Implementation

The attached demo_patient_cohort.csv provides a concrete 20-record example of the input schema. It contains a patient_name identifier plus clinical variables. The clinical columns contain numerical measurements, binary symptom indicators, gender, and smoking history.

Demo Cohort Check	Observed Value
Rows in supplied demo cohort	20
Clinical columns after excluding patient_name	46; the project README describes a 47-feature model space
Missing clinical values	0
Gender	12 Male, 8 Female
Smoking history	14 Never, 3 Current, 3 Former
Most frequent symptom indicator	Fatigue: 80% of demo records

The patient_name field is treated as a record identifier rather than a predictive clinical feature in this report. The project's documented 47-feature count refers to the clinical feature space used by the model.

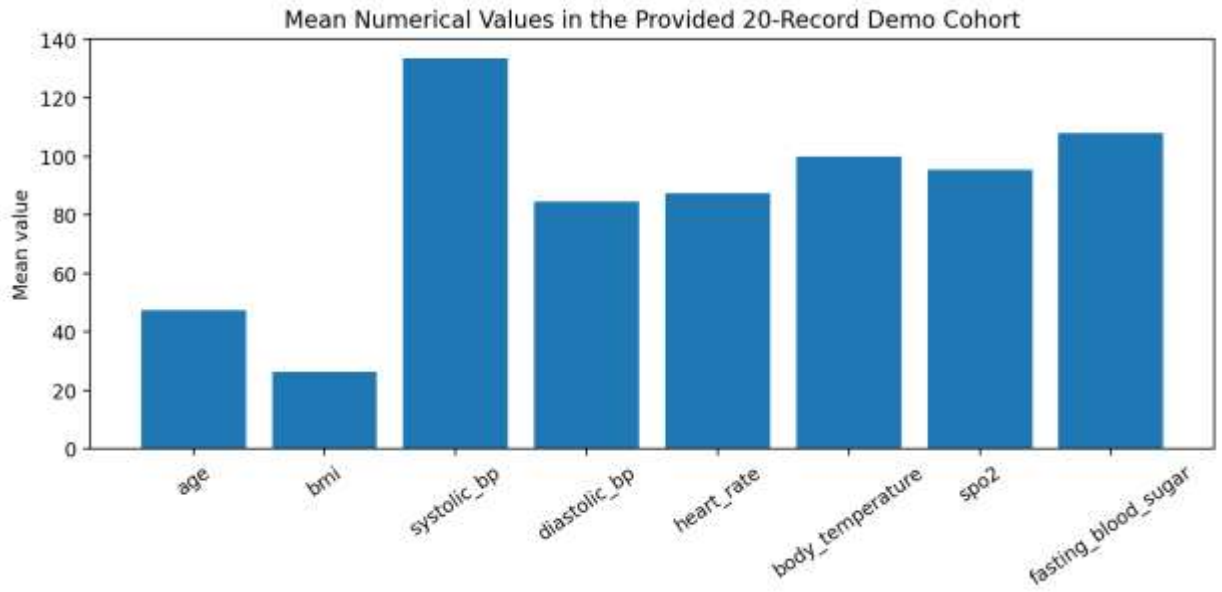


Figure 1. Mean values of the principal numerical variables in the supplied 20-record demonstration cohort.

3.5 Summary of EDA Findings

The demonstration cohort contains no missing clinical values and shows variation across age, body mass index, blood pressure, heart rate, temperature, oxygen saturation, and fasting blood sugar. The symptom indicators also vary substantially, with fatigue appearing most frequently in the supplied cohort. These observations illustrate why preprocessing and multi-feature classification are required.

No claim about the class balance of the full 5,400-record dataset is made here because the supplied case-study materials do not provide a complete class-frequency table. The README does state that the evaluation uses an independent stratified test set, supporting class-aware evaluation.

4. Algorithm Selection and Implementation

4.1 Why Multinomial Logistic Regression

The project presentation identifies three main reasons for choosing Multinomial Logistic Regression. Standard logistic regression is limited to binary outcomes; the multinomial extension handles multiple disease categories in one unified model; and the model produces probabilistic outputs, allowing alternative diagnoses to be considered.

- Naturally suited to multi-class classification.
- Produces a probability distribution across all disease classes.
- Coefficients can be inspected to understand feature influence.
- L2 regularization provides a mechanism to control model complexity.
- The approach integrates naturally with a web-based inference engine.

4.2 Mathematical Formulation

For K = 18 classes and D = 47 clinical features, the project computes a logit for each disease class:

```
z_k = w_kᵀx + b_k = Σ(j=1..D) w_kj x_j + b_k
```

The logits are converted into posterior probabilities using Softmax:

```
P(Y = k | x) = exp(z_k) / Σ(j=1..K) exp(z_j)
```

The documented training objective is categorical cross-entropy with an L2 penalty on the weights. The README also documents analytical gradients for the weights and bias and describes the use of Adam optimization.

4.3 Model Training and Regularization

The project implements the training process through a vectorized NumPy machine-learning core and a training orchestrator. The documented architecture uses L2 Ridge regularization and Adam optimization. The objective balances classification error against the magnitude of model weights.

```
J(W,b) = -1/N Σ_i Σ_k y_ik log(P_ik + ε)
        + (λ/2N) Σ_k Σ_j w_kj²
```

For interpretability, the project defines an odds-ratio transformation for a coefficient: OR_kj = exp(w_kj). An odds ratio greater than 1 indicates positive influence on the corresponding class, while a value below 1 indicates an opposing influence.

4.4 Evaluation and Benchmarking

The project reports evaluation on an independent stratified test set containing 1,080 patient cases. The tuned Multinomial Logistic Regression model is compared with K-Nearest Neighbors, Random Forest, and Gaussian Naive Bayes.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Macro ROC-AUC
Tuned Multinomial Logistic Regression	98.98%	0.9899	0.9898	0.9898	0.9999
K-Nearest Neighbors (k=7)	98.52%	0.9854	0.9852	0.9851	0.9996
Random Forest	96.94%	0.9698	0.9694	0.9695	0.9995
Gaussian Naive Bayes	96.11%	0.9620	0.9611	0.9612	0.9995

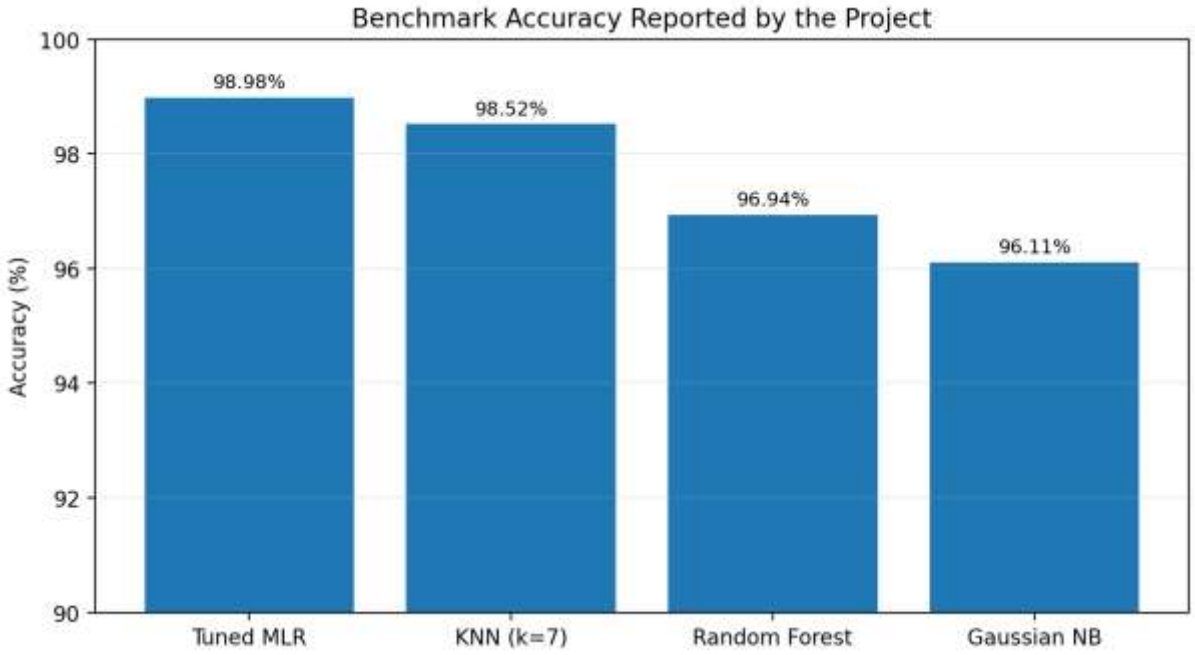


Figure 2. Accuracy comparison using the metrics reported in the project README.

According to the supplied results, tuned MLR is the highest-performing model among the four reported approaches on the stated test set. Its macro metrics are also close to its overall accuracy, indicating strong reported multi-class performance.

4.5 Explainability and Odds Ratios

Interpretability is a central design goal. The model exposes feature weights and converts them into odds ratios. The README's demonstration response for a patient with high fasting blood sugar and excessive thirst identifies those features as positive indicators for Type 2 Diabetes Mellitus.

5. System Design and Web Application

5.1 Architecture and Workflow

The presentation describes a sequential workflow in which preprocessing feeds feature selection, model training, and evaluation. The application then uses the trained inference engine to produce diagnostic output. The supplied app.py shows that Flask initializes a diagnostic engine and connects it to interactive pages and REST API endpoints.



Figure 3. Workflow reconstructed from the project presentation and implementation documentation.

5.2 Web Application Features

- Interactive patient diagnostic assessment page.
- Batch CSV upload and cohort prediction.
- Analytics dashboard using model metadata.
- Disease knowledge base with clinical information.
- About page describing the system and mathematical theory.
- Single-patient prediction with probability, differential diagnoses, explainability, triage information, and clinical guideline fields.

The application code defines routes for the main page, batch page, analytics page, knowledge base, and about page. The diagnostic engine is initialized when the Flask application starts. The main page receives symptom categories, numerical features, categorical features, and the disease list from the knowledge base.

5.3 REST API and Batch Prediction

The project exposes POST /api/predict for single-patient JSON input. A second endpoint, POST /api/batch-predict, accepts a CSV file, converts it into a Pandas DataFrame, performs batch prediction, and returns the total number of records, individual results, and a class-distribution summary.

```
POST /api/predict
Content-Type: application/json

POST /api/batch-predict
Multipart/form-data with a CSV file
```

The documented single-patient response can contain a top disease, confidence percentage, differential diagnoses, positive indicators, triage risk level, clinical warnings, specialist mapping, and recommended tests.

5.4 Technical Stack

Layer	Technologies / Tools
Programming Language	Python
Data Processing	NumPy, Pandas
Machine Learning	Scikit-learn; project-specific vectorized NumPy ML core
Web Framework	Flask with REST API
Visualization	Matplotlib, Seaborn; Chart.js in the web dashboard
Development Tools	Jupyter Notebook, PyCharm / Visual Studio Code
Version Control	Git / GitHub
Runtime	Python 3.9+; Windows, Linux, or macOS

6. Case-Study Analysis and Discussion

6.1 Example Diagnostic Scenario

The README provides a concrete single-patient demonstration. The input contains age, gender, smoking history, BMI, blood pressure, heart rate, body temperature, SpO2, fasting blood sugar, and symptoms including excessive thirst, frequent urination, and fatigue. The documented output identifies Type 2 Diabetes Mellitus as the top disease with a confidence of 99.89%.

```
fasting_blood_sugar → weight 2.41 → Increases Likelihood
excessive_thirst_polydipsia → weight 2.85 → Increases Likelihood
```

The documented response reports a MODERATE triage risk level and maps the case to an Endocrinologist / Diabetologist with recommended tests including HbA1c, fasting plasma glucose, and oral glucose tolerance testing. In a real setting, these outputs would be interpreted by a clinician rather than treated as definitive diagnoses.

6.2 Interpretation of Model Output

- Top diagnosis: disease class with the highest predicted posterior probability.
- Confidence percentage: human-readable top-class probability.
- Differential diagnoses: alternative classes remaining plausible under the model.
- Positive indicators: features whose learned weights increase likelihood of the predicted class.
- Triage information: decision-support risk category and warnings.
- Clinical guidelines: knowledge-base information such as specialist mapping and suggested tests.

6.3 Benefits

- High reported predictive performance on the project's independent stratified test set.
- A single model handles 18 disease categories.
- Probability outputs support differential reasoning.
- Coefficient-based explanations improve transparency.
- Batch prediction supports cohort-level analysis.
- Flask and REST endpoints make the model accessible through a practical software interface.
- The architecture is designed to scale toward larger datasets and clinical-platform integration.

6.4 Limitations and Responsible Use

The project presentation describes the solution as a foundation for clinical deployment, but the supplied materials do not document real-world prospective validation. High test-set accuracy is encouraging for the academic experiment but does not by itself establish safety, fairness, generalization across hospitals, or suitability for patient care.

- Validate against external and clinically representative populations before real-world use.
- Assess performance by disease class and relevant patient subgroups.
- Monitor calibration, drift, false negatives, and false positives after deployment.
- Protect patient data and implement appropriate authentication, authorization, and auditing.
- Keep a qualified clinician responsible for interpretation and final decisions.
- Use the system as decision support rather than an autonomous replacement for medical expertise.

7. Conclusion

This case study demonstrates an end-to-end machine-learning approach to multi-class disease diagnosis using Multinomial Logistic Regression. The project addresses a classification gap in which patient records contain multiple symptoms and measurements that must be compared across many possible disease categories. MLR provides a unified probabilistic framework that is both multi-class and interpretable.

The supplied project reports strong evaluation results, with the tuned MLR model achieving 98.98% accuracy and 0.9999 macro ROC-AUC on an independent stratified test set of 1,080 cases. The project also goes beyond model training by providing feature-level explainability, batch prediction, analytics, a disease knowledge base, and REST APIs through a Flask web application.

The case study shows how a statistical machine-learning model can be integrated into a practical software system for academic clinical decision-support research. Future work should focus on external validation, clinical evaluation, data governance, calibration, fairness, security, and safe human-in-the-loop deployment.

References / Source Materials

- blood_disease_project_no_gamma.pptx — project presentation supplied for the Machine Learning case study.
- Ml abstract.pdf — project abstract, introduction, software requirements, team and guide details.
- README(20261006-045939).md — project scope, disease classes, mathematical formulation, metrics, architecture, installation, and API documentation.
- app.py — Flask application routes, single-patient prediction, batch CSV processing, analytics and knowledge-base endpoints.
- demo_patient_cohort.csv — supplied 20-record demonstration cohort used for supplementary schema and descriptive checks.
- CaseStudy_Template-ML.docx — supplied case-study report structure and index.

Signature of the Team Members

_____ _____

M. Shiva **P. Bhavan**

Roll No.: 2520090135 Roll No.: 2520090146